

Azure Entra ID SSO Integration

End-to-End Enterprise Design & Rollout Strategy Document

Architecture: ASP.NET Core 8 MVC (UI) & Web API

Cloud: Azure App Services, Key Vault, Microsoft Graph

Methodology: Clean Architecture, Zero-Impact Feature Flagging

1. Executive Summary

This document outlines the technical architecture, implementation steps, and team rollout strategy for integrating Azure Entra ID (Single Sign-On) into a distributed .NET 8 enterprise application. The system is divided into two distinct components: an ASP.NET Core 8 MVC User Interface (UI) and an ASP.NET Core 8 Web API. Both applications adhere to strict Clean Architecture principles.

To support a seamless transition for an active development team of 4 engineers across 5 environments (Dev, Staging, UAT, PreProd, Prod), the implementation utilizes a **Zero-Impact Feature Flagging** strategy combined with App Service slot configurations.

2. Architecture Overview

2.1 Component Separation

- **UI Application (MVC):** Responsible for user interaction, Dual-Login routing (Local DB vs. SSO), and rendering views. It requests an access token via the Microsoft Authentication Library (MSAL) to act on behalf of the user when calling the API.
- **API Application (Web API):** Validates the incoming JWT bearer token and resolves the user's Object ID (OID). It handles business logic, database operations, and strictly enforces role-based access control (RBAC).

2.2 Secrets Management

Architectural Decision: Native Key Vault Provider

Custom "Key Vault Services" are classified as an anti-pattern for configuration. We utilize the native `Azure.Extensions.AspNetCore.Configuration.Secrets` provider. This injects Key Vault secrets directly into the `IConfiguration` dictionary at startup using Managed Identity, ensuring secrets are accessed exactly like local JSON settings without runtime retrieval overhead.

3. Implementation Guide

3.1 Shared Configuration (appsettings.json)

Both applications utilize feature flags to control authentication mechanisms without altering compiled code.

```
{
  "KeyVault": {
    "VaultUri": "https://kv-yourapp-dev.vault.azure.net/"
  },
  "AuthenticationFeatures": {
    "EnableSSO": false, // Disabled by default for local dev isolation
    "EnableLocalLogin": true
  },
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "Domain": "yourdomain.onmicrosoft.com",
    "TenantId": "YOUR_TENANT_ID",
    "ClientId": "YOUR_APP_CLIENT_ID"
  }
}
```

3.2 Bootstrapping Key Vault (Program.cs)

Required in the bootstrap of both the UI and API applications.

```
var builder = WebApplication.CreateBuilder(args);

var keyVaultUri = builder.Configuration["KeyVault:VaultUri"];
if (!string.IsNullOrEmpty(keyVaultUri))
{
    builder.Configuration.AddAzureKeyVault(
        new Uri(keyVaultUri),
        new DefaultAzureCredential()); // Uses Managed Identity or VS Creds
}
```

3.3 UI Solution Authentication

The UI is configured to support dual schemas, allowing simultaneous operations of existing forms authentication and new Entra ID SSO.

```
// UI Program.cs - Feature Flagged Authentication Pipeline
var enableSso = builder.Configuration.GetValue<bool>("AuthenticationFeatures:EnableSSO");

var authBuilder = builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
})
.AddCookie(CookieAuthenticationDefaults.AuthenticationScheme, options =>
{
    options.LoginPath = "/Account/Login";
});

if (enableSso)
{
    authBuilder.AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
        .EnableTokenAcquisitionToCallDownstreamApi(
            new string[] { builder.Configuration["DownstreamApi:Scopes"] })
        .AddDownstreamApi("MyApi", builder.Configuration.GetSection("DownstreamApi"))
        .AddMicrosoftGraph(builder.Configuration.GetSection("GraphApi"))
        .AddInMemoryTokenCaches();
}
```

3.4 API Database RBAC Mapping

To avoid disrupting existing database-driven RBAC mapping, the API implements an `IClaimsTransformation`. This ensures standard ASP.NET Core authorization attributes remain functional.

```

public class DatabaseRoleClaimsTransformation : IClaimsTransformation
{
    private readonly IUserRepository _userRepository;

    public DatabaseRoleClaimsTransformation(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public async Task<ClaimsPrincipal> TransformAsync(ClaimsPrincipal principal)
    {
        if (principal.HasClaim(c => c.Type == "RolesMapped"))
            return principal;

        var identity = new ClaimsIdentity(principal.Identity);

        // Extract Entra ID Object ID (OID)
        var oidClaim = principal.FindFirst("http://schemas.microsoft.com/identity/claims/objectidentifier")
            ?? principal.FindFirst("oid");

        if (oidClaim != null)
        {
            // Fetch Roles (Requester, Approver) based on Entra ID
            var dbRoles = await _userRepository.GetUserRolesByProviderIdAsync(oidClaim.Value);
            foreach (var role in dbRoles)
            {
                identity.AddClaim(new Claim(ClaimTypes.Role, role));
            }
        }

        identity.AddClaim(new Claim("RolesMapped", "true"));
        return new ClaimsPrincipal(identity);
    }
}

```

3.5 Microsoft Graph API Integration

Using the injected Graph service client, backend systems can automate Outlook email operations.

```

public class GraphEmailService : IEmailService
{
    private readonly GraphServiceClient _graphClient;

    public GraphEmailService(GraphServiceClient graphClient) { _graphClient = graphClient; }

    public async Task SendEmailAsync(string toEmail, string subject, string content)
    {
        var message = new Message
        {
            Subject = subject,
            Body = new ItemBody { ContentType = BodyType.Html, Content = content },
            ToRecipients = new List<Recipient> {
                new Recipient { EmailAddress = new EmailAddress { Address = toEmail } }
            }
        };

        await _graphClient.Me.SendMail.PostAsync(
            new Microsoft.Graph.Me.SendMail.SendMailPostRequestBody
            { Message = message, SaveToSentItems = true });
    }
}

```

4. Team Rollout & Deployment Strategy

To successfully integrate this feature within an active 4-developer team across 5 environments, a Trunk-Based (Dark Launching) deployment methodology is strictly enforced.

4.1 Local Developer Isolation

Developers will continue using the existing authentication methods without disruption. Create an `appsettings.local.json` file excluded via `.gitignore` for the specific developer testing SSO locally.

Developer Workflow

Your team members will pull the updated codebase. Because the shared `appsettings.Development.json` has `EnableSSO` set to `false`, the Entra ID middleware will not execute, and their local databases and login screens will function exactly as they did before the integration. You will use your isolated `appsettings.local.json` mapped to your localhost App Registration.

4.2 Multi-Environment App Service Slots Rollout

Azure App Service Configuration "Sticky" Slot Settings will govern the deployment rollout. You will configure the `AuthenticationFeatures:EnableSSO` App Setting dynamically per environment.

Environment	Slot Configuration	EnableSSO	EnableLocal	Purpose
Dev	dev-slot	True	True	Active validation of Azure-hosted end-to-end Entra ID flow.
Staging	staging-slot	True	True	Cross-team integration and QA testing phase.
UAT	uat-slot	False	True	Client acceptance testing (SSO off until client readiness).
PreProd	preprod-slot	False	True	Dry-run staging for production configurations.
Prod	prod-slot	False	True	Live environment. Flipped to True dynamically on launch day.

Go-Live Procedure

When it is time to enable SSO in Production, **no new code deployment is required**. The lead engineer updates the Azure App Service Configuration variable `AuthenticationFeatures:EnableSSO` to `true` and restarts the App Service. In the event of an authentication failure, the value can be instantly toggled to `false` to invoke an immediate rollback.